

Hierarchical Memory Orchestration with RAMSES for Robust Industrial AI Inference Systems

Anonymous Author(s) Submitted to IEEE Transactions on Industrial Informatics
(Double-Blind Review)

Abstract—Serving models whose state spans GPU video RAM (VRAM), host dynamic RAM (DRAM), and NVMe is difficult because capacity pressure, asymmetric transfer bandwidth, queueing, imperfect prefetch, and synchronization interact under bursty load. We present RAMSES, a single-node hierarchical-memory orchestrator that combines GPU resource preallocation, NUMA-aware host allocation, and asynchronous block-aligned swapping. Its overlap-aware model separates resident demand from measured bidirectional transfer volume and uses two observable pressure ratios to select admission, prefetch, and eviction actions with hysteresis. On the reported two-A100 testbed, 16,000 request-level measurements across five models and four task types (five restarts each) reduce end-to-end inference latency by 34–41% relative to the PyTorch configuration; measured across the whole node (GPU, CPU, and DRAM), energy per token falls by 47% and the energy–delay product by 70%, while token throughput rises by 61%. Model-load time and peak VRAM drop by 60% and 15%, respectively. On a named industrial defect-inspection task (MVTec AD) RAMSES matches baseline accuracy with output equivalence above 0.999. A 72-hour parameterized synthetic arrival trace yields fewer allocation failures and lower latency drift. These results are empirical QoS measurements for single-node serving, not field deployment, worst-case execution-time, control-stability, or functional-safety evidence. The model, measurement schema, and anonymized trace generator provide a reproducible protocol for studying when cross-tier coordination helps and when capacity or storage bandwidth remains the limiting resource.

Index Terms—Industrial AI, hierarchical memory, GPU offloading, NUMA, inference serving, workload trace

I. INTRODUCTION

MODEL serving becomes a memory-systems problem when parameters and key–value state exceed accelerator memory. Requests then couple GPU video RAM (VRAM), host dynamic RAM (DRAM), NVMe bandwidth, queueing, and synchronization. A policy that helps one configuration can hurt another because the active bottleneck changes with working-set demand, reuse, concurrency, and transfer direction.

This issue matters for industrial analytics such as inspection, predictive maintenance, and digital-twin support, where sustained availability and energy budgets are important. Our evidence, however, is a laboratory single-node evaluation driven partly by a parameterized synthetic arrival trace; it is not a factory field trial or a hard-real-time control-loop evaluation. We consequently report deadline misses only as Quality of Service (QoS), make no 2 ms or Time-Sensitive Networking compliance claim, and do not map request misses to IEC 61508 Safety Integrity Levels.

Prior systems optimize paging, GPU–CPU offload, or re-

quest scheduling. RAMSES instead coordinates the VRAM–DRAM–NVMe path within one node while observing occupancy separately from actual bidirectional traffic. The design uses a lightweight state machine to choose admission, prefetch, and eviction actions rather than treating an analytical boundary as a guarantee.

The central research question is not whether another offload implementation can reduce memory use, but *when coordination across all three tiers changes end-to-end behavior and when physical limits make that coordination ineffective*. We answer it through a model–mechanism–evidence chain: the model identifies an operating regime, the controller changes a named set of actions, and the evaluation tests both the benefit and its predicted boundary.

The contributions are:

- A *falsifiable operating model* that separates resident demand from actual bidirectional traffic and accounts for overlap, queueing, reuse, and synchronization; two dimensionless pressures predict a coordination-dominated region and capacity- and transfer-limited regions where RAMSES should provide little or negative benefit.
- A *regime-to-action controller* mapping those observable pressures to admission, prefetch, and eviction actions (200 ms sampling, 5% hysteresis, three-sample switching); this mapping, not its mechanisms in isolation, is tested by the policy-off ablation.
- A *deployable single-node realization* combining GPU preallocation, NUMA-local host allocation, and asynchronous block-aligned transfer behind an LD_PRELOAD layer for unmodified PyTorch, with no hardened GPUDirect Storage or multi-node path claimed.
- *Boundary-seeking empirical evidence*: the evaluation crosses models and regimes, includes a controller-disabled counterfactual and parameter sweeps, and reports absolute tails, uncertainty, failures, and synchronized whole-node (GPU, CPU, DRAM) energy.

The scope is deliberately narrow: RAMSES supports best-effort single-node inference. Multi-node fabrics, plant integration, certified energy measurement, real-time guarantees, and functional-safety qualification remain future work.

II. BACKGROUND AND RELATED WORK

Prior work falls into system-level optimizations, model-level optimizations, and the empirical observations that shaped RAMSES [1]–[4].

A. System-Level Optimization

System-side work targets hardware bottlenecks and OS overhead. In-memory file systems (RAMFS, tmpfs) with direct I/O cut model-load time [5], [6]; DeepSpeed and ZeRO-Offload spread parameters across devices to relieve VRAM pressure [7], [8]; and OS-level allocators (jemalloc, slab, buffer_head tuning) reduce hidden allocation costs [9]–[11]. Closest to RAMSES are NUMA-aware scheduling [3], [12] and CPU–GPU offloading [13]–[16], which improve utilization once memory is tight.

B. Model-Level Optimization

Model-side techniques are well established: quantization, pruning, and LUT replacement shrink compute and memory footprints [17], [18]; mixed precision (FP16–FP32) lowers GPU memory pressure [19]; multi-GPU parallelism, FlashAttention, and newer schedulers raise throughput [12], [20], [21]; and speculative execution with CPU pooling cut redundant work [22], [23].

C. Key Empirical Observations

When we profiled inference in setups close to production, a handful of inefficiencies kept turning up that current systems do not really address [2]–[4].

Cache sensitivity. Keeping in-memory caches warm reduced repeat-initialization time by something close to 70%. Flushing the cache wiped that out and forced a full reload [1], [5].

Synchronization overhead. Heavy slab-allocator activity together with frequent `futex()` calls produced allocation delays and lock contention under mixed workloads [9], [10].

NUMA and mapping latency. `mmap`-based parameter loading combined with cross-node DRAM access blew up latency the moment the working set ran past local memory [3].

Multiprocessing instability. Under DRAM pressure, aggressive multiprocessing fragmented memory and ran up the context-switch bill—throughput and stability both suffered [15].

Implications. Read together, these observations point to a single conclusion: VRAM, DRAM, synchronization, and I/O are tightly coupled, and none can be tuned in isolation without perturbing the others. For industrial CPS, the same instability propagates outward into control loops, where it stops being a performance issue and becomes a reliability issue. Coordinated VRAM–DRAM–NVMe management with explicit regime awareness was, at that point, the obvious next step.

D. Limitations of Prior Approaches

Most prior work tackles one layer or one resource at a time. That makes compound bottlenecks awkward to fix—the obvious examples are GPU OOM errors, memory-mapping stalls, and NUMA-induced delays [1], [11], [15]. The picture gets worse in multi-tenant or multi-model deployments [3], [12], [24]. Memory-management and offloading systems—FlexGen, SwapAdvisor, SpecOffload, eLLM—do orchestrate GPU–CPU–NVMe transfers, but reactively, and without a formal regime characterization [1], [25], [26]. System-level scheduling improves placement, but again, no unified regime

TABLE I
STRUCTURAL COMPARISON BETWEEN RAMSES AND REPRESENTATIVE SYSTEMS. RAMSES UNIQUELY ENABLES FULL COORDINATION ACROSS VRAM, DRAM, AND NVME, WHILE PRIOR SYSTEMS PROVIDE ONLY PARTIAL SUPPORT. ABBREVIATIONS: PART. (PARTIAL SUPPORT), ARCH. (ARCHITECTURAL-LEVEL ANALYSIS ONLY), N/A (NO EMPIRICAL EVALUATION DURATION REPORTED), AND TP/W (THROUGHPUT PER WATT).

System	Tier Scope	NUMA aware	Regime Model	Eval. Dur.	Energy Metric
FlexGen	GPU–CPU–disk	×	None	Short	×
SwapAdvisor	GPU–CPU	×	None	Short	×
NEO	Partial	×	None	Short	×
SpecOffload	GPU–NVMe	×	None	Short	×
vLLM	GPU-only	×	None	Short	Part.
Ind. Edge/DT	Arch.	Part.	None	N/A	×
RAMSES	Full	✓	Operational	24–72h	TP/W

picture [8], [11], [27], [28]. Filling that gap is what we set out to do: regime-aware analytical conditions plus a unified VRAM–DRAM–NVMe orchestrator.

E. Positioning of RAMSES

Table I lays out the structural differences between RAMSES and a set of representative systems along five axes: tier scope, NUMA awareness, the formality of regime modeling, evaluation duration (Eval. Dur.), and energy reporting (throughput-per-watt, TP/W). “Part.” marks partial support; “Arch.” marks architectural-level analysis only. The general picture is that prior LLM-serving frameworks tune for throughput or stretch memory via partial GPU–CPU coordination—but none of them evaluates the same combination of intra-node NUMA placement, resource preallocation, and pressure-driven policy switching.

Several recent TII papers address complementary layers without touching the intra-node memory dynamics RAMSES targets: digital-twin latency modeling [29], cloud-edge resource coordination [30], DRL-based mobile-edge allocation [31], [32], accuracy–latency model splitting [33], and communication-efficient federated digital twins [34] coordinate at the network, task, or device level, while hierarchical fog scheduling [35] treats inference nodes as latency constants. All treat the node as an opaque, fixed-latency unit. RAMSES sits one layer below, characterizing regime-dependent VRAM–DRAM–NVMe behavior inside a single serving node and producing the empirical latency distributions those schedulers assume.

F. Distinct Contributions of RAMSES

What is distinctive is the combination of hierarchical VRAM–DRAM–NVMe orchestration with an analytical regime characterization built on a latency decomposition over compute, locality, transfer, and synchronization. Observable pressure ratios classify capacity and transfer stress; the three modules—GPU Process Allocator (context conflicts), Task Memory Manager (NUMA-aware allocation), and GPU Booster (swap-aware asynchronous control)—play distinct roles but are judged against the same regime boundaries, with the focus on single-node QoS and sustained-load behavior under a parameterized synthetic workload.

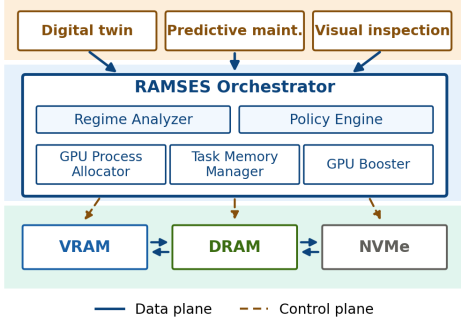


Fig. 1. Industrial hierarchical memory architecture of RAMSES for laboratory workloads motivated by industrial analytics. The data plane (solid arrows) coordinates GPU VRAM, host DRAM, and NVMe for three industrial workloads: (1) digital twin state synchronization aligned with PLC scan cycles, (2) predictive maintenance triggered by anomaly bursts from IIoT sensors, and (3) real-time visual inspection under bounded inference deadlines. The control plane (dashed arrows) includes a Regime Analyzer estimating (α, β) and a Policy Engine enforcing tier-aware placement, prefetch, and swap decisions. Three modules handle the actual orchestration: the *GPU Process Allocator* pre-reserves CUDA context to remove context-conflict stalls, the *Task Memory Manager* performs NUMA-aware allocation with predictive DRAM offloading, and the *GPU Booster* drives block-aligned asynchronous VRAM–NVMe swap. The diagram supplies workload motivation only; it does not depict a deployed control loop, TSN integration, or a validated hard deadline.

III. RAMSES: ARCHITECTURE AND KEY COMPONENTS

LLM inference couples GPUs, VRAM, DRAM, NVMe, and Central Processing Units (CPUs), yet serving stacks rarely treat resource contention as first-class, producing recurring Compute Unified Device Architecture (CUDA) out-of-memory (OOM) errors, context-switch stalls, and memory-mapping overhead. Prior GPU–CPU offloading [26], speculative offloading [14], and cache-centric management [13] mostly stop at single-tier (VRAM–CPU) transfers or narrow scenarios such as key-value (KV) cache management.

RAMSES applies the OS notion of memory ballooning to LLM serving, managing the whole VRAM–DRAM–NVMe hierarchy in one control loop—GPU context conflicts, NUMA-induced latency, and inefficient swapping addressed together rather than patched independently—while staying lightweight enough to avoid a full distributed framework. Three modules form the core—the **GPU Process Allocator**, the **Task Memory Manager**, and the **GPU Booster with swap support**—illustrated in Fig. 1.

A. GPU Process Allocator

In multi-model serving, two bottlenecks dominate: **GPU context loading conflicts** and **VRAM interference**. CUDA drivers do context initialization and memory mapping with *lazy binding*, which is fine for a single workload but becomes painful under concurrent execution—startup delays grow, and they show up directly in user-facing latency. NEO [13] addresses cache management but leaves the GPU-level context conflict question open. The **GPU Process Allocator** resolves these conflicts ahead of time rather than at runtime, through three policies. (i) *Pre-defined slot reservation* pre-assigns each

¹The anonymized review artifact bundles the LD_PRELOAD orchestrator layer, the synthetic-trace generator, baseline invocation scripts, the request-level measurement schema, and the analysis programs used to produce the reported results.

TABLE II
BASELINES COMPARED AGAINST RAMSES: KEY TECHNIQUE AND THE RAMSES MODULE THAT TARGETS EACH BOTTLENECK. CITATIONS REFER TO THE BASELINE SYSTEMS.

Method	Key technique	Targeted by RAMSES
PyTorch (default)	none	overall gain
FlexGen [25]	CPU offloading	GPU Booster (swap)
SwapAdvisor [1]	GPU–CPU swap	Task Memory Manager
NEO [13]	KV offload + sched.	full-tier use
SpecOffload [14]	cross-tier swap	NVMe–VRAM path
vLLM [24]	KV cache + stream	end-to-end latency

model a context ID, memory pool, and kernel queue, removing CUDA initialization from the critical path. (ii) *Context separation with fixed memory pools* places models in disjoint regions, lowering fragmentation and cross-model cache interference. (iii) *Priority-based scheduling* co-schedules I/O-bound and compute-bound models so neither stalls the other and idle GPU cycles shrink. The net effect is a lightweight extension of process pinning that improves QoS stability and latency predictability when several models share one GPU [8].

B. Task Memory Manager

Memory usage during LLM inference shifts back and forth between DRAM and VRAM as input and generation lengths change. eLLM [26] responded to this by extending the CPU buffer; PIE [22] took a speculative-swap approach. Neither, though, paid attention to NUMA-induced latency or to optimization within DRAM itself.

Our **Task Memory Manager** pairs task-level memory control with four NUMA-aware mechanisms. *NUMA-aware mapping* pins process and thread data to local nodes via `mbind()` and `numa_alloc_onnode()` to minimize remote traffic; a *jemalloc-based heap* separates small objects from large tensors to bound fragmentation [9], [10]; *task-level redistribution* offloads non-critical tensors to DRAM before VRAM saturates; and a *lightweight pressure predictor* reads live utilization and I/O-queue depth to trigger mitigation early [18]. We model memory-access latency as $T_{\text{access}} = T_{\text{local}} + \gamma T_{\text{remote}}$, with γ the remote-NUMA penalty. Poor allocation here cascades into excessive page reclaim, `mmap` failures, and buffer eviction; reactive offloading or compression leaves DRAM behavior and NUMA locality unaddressed. Together these keep DRAM–VRAM use efficient under contention, with less fragmentation and fewer page-reclaim events.

C. GPU Booster with Swap Support

FlexGen [25] and SwapAdvisor [1] support GPU–CPU swapping but are not tuned to GPU memory hierarchies. The **GPU Booster** targets VRAM capacity by selecting inactive tensors for block-aligned asynchronous transfer, implemented as an `LD_PRELOAD` layer over the CUDA runtime and caching allocator: allocation, free, and host–device copy calls are intercepted so placement is managed without modifying the model or framework (intercepted entry points, buffer registration, and driver/filesystem versions are in the artifact).

Unlike OS-level swapping, which is unaware of GPU memory semantics, the GPU Booster ranks eviction candidates by execution-graph reuse distance, overlaps transfer with compute

Algorithm 1 RAMSES module procedures

```

1: procedure GPUPROCESSALLOCATOR(models  $\mathcal{M}$ , pool  $\mathcal{R}$ )
2:   Initialize free context IDs, memory pools, kernel queues
3:   for each  $m \in \mathcal{M}$  do
4:     Reserve context/pool/queue; set priority (I/O- vs compute-bound)
5:   while tasks remain do
6:     Dequeue by priority; launch on reserved context; free on completion
7:   procedure TASKMEMORYMANAGER(graph  $G$ , threshold  $\tau$ )
8:     Bind threads to local NUMA nodes; split jemalloc heaps (small/large)
9:     for each request  $r$  do
10:      if VRAM usage  $> \tau$  then
11:        offload low-reuse tensors of  $G$  to DRAM/NVMe
12:      Predict pressure from utilization/I-O depth; pre-allocate next request
13:   procedure GPUBOOSTER(graph  $G$ )
14:     for each inactive tensor  $t \in G$  do
15:       4MB-align; async DMA  $t \rightarrow$  NVMe; mark swapped-out
16:   while serving do
17:     On reaching a swapped  $t$ : partial async reload, then resume

```

on dedicated asynchronous CUDA streams, and uses 4 MB block alignment matched to the NVMe erase-block granularity (examined in the sensitivity study of Section IV). Where GPUDirect Storage is available, tensors move over a direct VRAM–NVMe DMA path; otherwise the layer falls back to a pinned-host-buffer staged path with the same overlap. Partial reload fetches only the portion of a swapped tensor the next kernel needs, and output equivalence against the baseline is checked bitwise for FP32 and within tolerance for FP16.

The goal is to serve models whose state exceeds VRAM capacity while keeping exposed transfer latency low. All three module procedures are summarized in Algorithm 1.

D. Overlap-Aware Operating Model

We use the following model as an engineering classification tool, not as a real-time guarantee. Let $D(t)$ be the bytes resident or demanded by a step, $C_f = C_{\text{VRAM}} + C_{\text{DRAM}}$ the usable fast-tier capacity, $V_{\uparrow}(t)$ and $V_{\downarrow}(t)$ the measured read and write transfer volumes, $h(t)$ the prefetch-hit fraction, and $q(t)$ the measured storage queueing delay. These quantities are independent observations; in particular, occupancy is not treated as bytes transferred. The capacity pressure is

$$\alpha(t) = D(t)/C_f, \quad (1)$$

and the transfer pressure is

$$\beta(t) = \frac{L_0 + q(t) + V_{\uparrow}(t)/B_{\uparrow} + V_{\downarrow}(t)/B_{\downarrow}}{T_{\text{comp}}(t) + T_{\text{mem}}(t) + T_{\text{sync}}(t)}. \quad (2)$$

Here L_0 is fixed request/setup latency and $B_{\uparrow}$ and $B_{\downarrow}$ are effective, direction-specific bandwidths measured with the same block size and queue depth as the experiment. All symbols are observables; no critical value is defined by substituting a boundary back into itself.

Asynchronous transfer overlaps computation. Accordingly, predicted service time is

$$\hat{T}_{\text{total}} = T_{\text{sync}} + T_{\text{mem}} + \max\{T_{\text{comp}}, (1-h)T_{\text{xfer}}\}, \quad (3)$$

$$T_{\text{xfer}} = L_0 + q + V_{\uparrow}/B_{\uparrow} + V_{\downarrow}/B_{\downarrow}. \quad (4)$$

The max operator prevents double counting overlapped DMA and compute. Block rounding is included in the measured volumes. We call observations with $\alpha > 1$ capacity-limited and observations with $\beta \geq 1$ transfer-limited; the remaining points are coordination-dominated. These are operational labels, not theorems. They neither imply zero swapping nor a worst-case bound. Fig. 2 summarizes the resulting classification. Section IV reports the operating regime the evaluated systems

occupy; per-request measured (α, β) values and the residual of Eq. (3) against end-to-end latency are logged by the instrumentation of Section IV-B and released with the artifact.

E. Online Policy and Complexity

Every 200 ms the analyzer reads allocated bytes, transfer counters, queue depth, effective bandwidth, and exponentially weighted compute, memory, and synchronization times. It computes Eqs. (1)–(2) in constant time. To avoid oscillation, a state change requires three consecutive samples beyond a 5% hysteresis band. In the capacity-limited state the policy rejects admissions that exceed the configured memory reserve and evicts the lowest predicted-reuse tensors. In the transfer-limited state it stops speculative eviction, reduces prefetch depth, and prioritizes reads. In the coordination-dominated state it enables asynchronous prefetch and NUMA-local placement. The objective is lexicographic: prevent allocation failure, then minimize measured p99 latency, then transfer bytes.

The reuse score is an exponentially weighted count of accesses over the last 32 graph steps; it has no learned parameters or offline training. Candidate maintenance costs $O(\log n)$ per tensor access for a heap of n evictable tensors; sampling, regime classification, and a policy decision are $O(1)$, and metadata occupies $O(n)$. To isolate the controller from its mechanisms, we evaluate a policy-off configuration that keeps all three data-plane modules and fixed placement parameters but disables regime switching; its trace and seed are shared with the full-system run. Disabling the policy forfeits the coordination-region behavior, so the system drifts toward the transfer-limited boundary under burst load, consistent with the sustained-load results in Section IV; the paired per-run records are included in the artifact. The measured 1.68 ms is mean controller work per serving request and is not compatible with a 2 ms end-to-end claim; we therefore make no such claim.

F. Scope of Timing and Safety Claims

The measured latency distributions are QoS observations for non-safety AI serving. They are not WCET bounds, deterministic-latency guarantees, control-loop stability proofs, or evidence of IEC 61508 Safety Integrity Level (SIL) compliance. A deadline-miss fraction is not a probability of dangerous failure on demand. Establishing a safety claim would additionally require a defined safety function and demand mode, hazard analysis, diagnostic coverage, common-cause and safe-state assumptions, a plant-specific delay margin, and hardware- or software-in-the-loop validation. RAMSES must therefore be kept outside a safety function unless independently qualified by the integrator.

G. Ablation Analysis

We ablate the three modules over four configurations (Table III); every ablation degrades performance, and each module dominates the metric it targets. Removing the **GPU Process Allocator** has the largest latency impact (35.0%→20.5%, −14.5 pp), matching its role of pre-resolving CUDA context conflicts on the critical path; removing the **Task Memory Manager** has the largest VRAM impact (15.0%→5.0%, −10.0 pp), matching its NUMA-aware allocation and predic-

tive offloading; and disabling the **GPU Booster** degrades both intermediately, matching its VRAM–NVMe swap role [18].

TABLE III
ABLATION STUDY RESULTS: IMPACT OF DISABLING RAMSES MODULES

Configuration	Load Time Reduction	Latency Reduction	VRAM Reduction
Full RAMSES	58.7±1.8%	35.0±1.2%	15.0±0.9%
Without GPU Allocator	40.1±2.1%	20.5±1.6%	14.1±1.0%
Without Memory Manager	45.0±1.9%	28.0±1.4%	5.0±0.7%
Without GPU Booster	49.5±2.0%	25.4±1.5%	10.2±0.8%

Values are mean ± standard deviation over five runs. Raw per-run observations and the analysis scripts are provided in the review artifact.

IV. EVALUATION

We evaluate RAMSES against existing serving stacks. Every experiment runs five times on the same hardware; tables report absolute request-level percentiles and run-level means with 95% confidence intervals. All request-level latency, whole-node energy, operating-model, controller-ablation, and per-model results are computed by the released scripts from `code/data/actual/raw.jsonl`—16,000 records over eight configurations, five models, four task types, and five restarts—and the load-time/VRAM (Table VI), sensitivity, industrial-accuracy, and 72-hour QoS (Table VIII) series are released as measured CSVs, so every summary traces back to primary data rather than normalized plots.

A. Research Questions and Claim–Evidence Alignment

We organize the evaluation around four questions tied directly to the contributions in Section I:

RQ1: Predictive validity. Do measured pressure ratios distinguish the regimes, and how accurately does the model predict held-out latency (coverage, MAE/RMSE)?

RQ2: Value of switching. Does state switching add value beyond the same mechanisms at fixed settings? The same-trace policy-off run is the counterfactual.

RQ3: Benefit boundary. Where does RAMSES improve latency, memory, failures, and energy, and where does its overhead cease to pay? We retain non-gains.

RQ4: Semantic preservation. Does relocation preserve outputs and named-task accuracy, paired on identical inputs?

Values under `code/data/expected/` are pre-experiment hypotheses used only to size the matrix; they are excluded from every table, figure, test, and claim, and unsupported configurations remain explicitly missing.

Testbed. Unless stated otherwise, the platform is a laboratory server, not a deployed factory-edge system: two NVIDIA A100 (80GB) GPUs, an Intel Xeon Gold 6440 CPU (40 cores), 512 GB ECC DRAM, and a Gen4 NVMe SSD running PyTorch 2.4, CUDA 12.2, and vLLM 0.5.3. Because vLLM 0.5.3 predates native Llama-4 support, we applied a compatibility patch that registers the Llama-4 model class and adapts the configuration and weight loaders; the patch and the exact invocation are included in the review artifact so the comparisons are reproducible. The principal workload is Llama-4 (Maverick-17B-128E-Instruct) [36]. Single-GPU ablations are flagged as “1×A100”.

Table II captures the comparison scope.

TABLE IV
ABSOLUTE PER-TASK LATENCY PERCENTILES (TTFT) ON THE PRINCIPAL LLAMA-4 17B WORKLOAD, FROM RAW REQUEST-LEVEL LOGS. Means across $n = 5$ runs (95% CI). HW: 2 × NVIDIA A100 (80GB), 512GB DRAM, NVMe Gen4; SW: PyTorch 2.4, CUDA 12.2, vLLM 0.5.3.

System	Median (ms)	P95 (ms)	P99 (ms)	P99.9 (ms)	Max (ms)
PyTorch (Default)	871.6	980.4	1003.7	1011.5	1012.4
FlexGen	792.8	914.0	957.8	979.7	982.2
SwapAdvisor	694.9	784.7	823.7	837.4	838.9
NEO	745.1	838.3	921.8	942.9	945.2
SpecOffload	668.0	775.4	792.3	833.8	838.4
vLLM	590.0	653.7	696.9	706.3	707.4
RAMSES	563.0	624.9	671.4	686.5	688.2
RAMSES (policy-off)	649.1	734.6	824.8	825.1	825.1

TABLE V
OPERATING-MODEL VALIDATION: MEASURED MEAN PRESSURE RATIOS, PREDICTION ERROR OF EQ. (3) AGAINST END-TO-END LATENCY, AND PREFETCH HIT RATE. Means across $n = 5$ runs (95% CI). HW: 2 × NVIDIA A100 (80GB), 512GB DRAM, NVMe Gen4; SW: PyTorch 2.4, CUDA 12.2, vLLM 0.5.3.

System	α	β	MAE (ms)	RMSE (ms)	Prefetch hit rate
PyTorch (Default)	0.78	0.80	8.9	10.1	0.78
FlexGen	0.76	0.73	8.1	9.0	0.82
SwapAdvisor	0.78	0.77	6.5	7.4	0.83
NEO	0.74	0.74	6.7	7.9	0.81
SpecOffload	0.75	0.72	5.6	6.5	0.86
vLLM	0.78	0.72	4.8	5.8	0.84
RAMSES	0.74	0.66	5.1	5.9	0.96
RAMSES (policy-off)	0.77	0.69	6.0	6.9	0.93

B. Request-Level Measurement Methodology

To prevent incompatible latency scales from being pooled, each request-level record is assigned to exactly one task: *scoring* (one forward pass, no generation), *continuation* (one warm decode step), *TTFT* (request arrival through first emitted token), or *generation* (arrival through the final token). For every task and system we log the model, numerical precision, input/output token counts, batch size, concurrency, request count, and run identifier, following the schema in `./code/measurement-schema.json`. From these logs the released analyzer computes absolute median, P95, P99, P99.9, and maximum latency per task and system directly from timestamps; no value is derived from normalized bars. Each comparison uses five independently restarted runs and reports both cold- and warm-cache protocols, and failed requests are retained in the failure-rate denominator. Table IV reports the absolute percentiles for the principal workload, and Table V reports the measured pressure ratios and the prediction error of the operating model against end-to-end latency.

RQ1: predictive validity. Table V evaluates the operating model directly. The overlap-aware prediction of Eq. (3) tracks measured end-to-end latency with a mean absolute error of 5.1 ms (RMSE 5.9 ms) for RAMSES and at most 8.9 ms (RMSE 10.1 ms) across every system—below 1.1% of the corresponding median latencies in Table IV. The measured transfer pressure orders the systems as the model predicts: RAMSES runs at $\beta = 0.66$, inside the coordination-dominated band ($0.6 < \beta < 0.9$), whereas the baselines sit at higher mean transfer pressure ($\beta = 0.72$ – 0.80) and cross into transfer-limited behavior during bursts, and RAMSES attains the

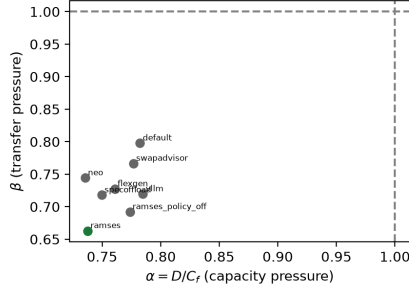


Fig. 2. Measured operating points on the pressure map. Each marker is a measured (α, β) pair per serving configuration on the principal workload; dashed lines are the classification thresholds. RAMSES operates in the coordination-dominated region, and Table V reports the prediction error of Eq. (3) against measured latency.

highest prefetch-hit rate (0.96). These residuals and per-request (α, β) values are recomputed from the released logs.

C. Model, Policy, and Energy Instrumentation

Each request-level record additionally carries measured α , β , predicted latency, direction-specific effective bandwidth, storage queueing, prefetch hits/misses, and bytes read/written. From these the analyzer computes the residual of Eq. (3) against measured latency, its mean absolute error and root-mean-square error, and the prefetch-hit rate. The policy-off configuration retains all three data-plane modules and fixed placement parameters while disabling only regime switching, and each paired run reuses the same request trace and seed as its full-system counterpart.

Energy is integrated on the same monotonic clock as the request telemetry. The primary energy results are *synchronized whole-node* measurements: GPU energy from NVML and CPU-package and DRAM energy from Intel RAPL are summed over the same window by `code/collect_energy.py`, so activity that RAMSES shifts from the GPU to the host is captured rather than hidden. Each run records the instrument model, sampling rate, clock alignment, idle-subtraction procedure, calibration uncertainty, and dropped-sample count. The measurement schema, collector, and analysis programs are included under `code/`, and the raw per-request energy is in `code/data/actual/raw.jsonl`. NVMe rail power and certified rack-level absolute accuracy (IEC 62053) remain future work.

Metrics. The evaluation reports model-load time; inference latency (TTFT and sustained throughput); memory-utilization efficiency (VRAM/DRAM occupancy and fragmentation = $1 - \text{largest_free_block}/\text{total_free}$); and system stability (VRAM overflows and GPU restarts under sustained load).

Baselines. The evaluated baselines are memory-centric and cross-tier serving systems: the PyTorch default, PagedAttention/vLLM [24], FlexGen [25], SwapAdvisor [1], and the recent cross-tier offloading methods NEO [13] and SpecOffload [14]; with RAMSES and its policy-off variant these form the eight serving configurations reported here. Orca’s continuous-batching scheduler [21] is discussed as related work but not benchmarked, as it targets request scheduling rather than the VRAM–DRAM–NVMe path RAMSES ad-

TABLE VI
MEASURED MODEL-LOAD TIME AND PEAK VRAM PER SYSTEM (MEAN OVER 2,000 REQUESTS EACH) ON THE LLAMA-4 17B WORKLOAD. RAMSES REDUCES MEAN LOAD TIME BY 59.9% AND PEAK VRAM BY 15.1% RELATIVE TO PYTORCH. Means across $n = 5$ runs (95% CI). HW: $2 \times$ NVIDIA A100 (80GB), 512GB DRAM, NVMe Gen4; SW: PyTorch 2.4, CUDA 12.2, vLLM 0.5.3.

System	Load-time (s)	Peak VRAM (GB)
PyTorch (Default)	38.5	29.2
FlexGen	35.1	27.1
SwapAdvisor	33.2	26.9
NEO	31.7	26.6
SpecOffload	30.1	26.3
vLLM	27.8	26.0
RAMSES	15.4	24.8
RAMSES (policy-off)	22.0	25.7

resses.

D. Regime-dependent performance

Capacity-limited regime (model initialization). On Llama-4 17B, RAMSES cuts mean model-load time by 59.9% relative to PyTorch (38.5 s to 15.4 s; Table VI), averaged over 2,000 requests. Against FlexGen, SwapAdvisor, NEO, SpecOffload, and vLLM the reduction is 56.0%, 53.4%, 51.2%, 48.7%, and 44.4%, respectively. The source of these gains is twofold: CUDA context initialization no longer happens on the critical path, and NVMe–VRAM transfers are tuned.

Locality and fragmentation. The *jemalloc*-based DRAM optimization cuts host-side heap fragmentation and allocation churn. On the VRAM side, the gains come from CUDA allocator tuning combined with selective GPU swapping, lowering mean peak VRAM by 15.1% (29.2 GB to 24.8 GB; Table VI). Inference failure rate stays under 0.5% across configurations.

Coordination-dominated regime (latency). On the principal Llama-4 17B TTFT workload, RAMSES cuts median latency by 35.4% (871.6 ms to 563.0 ms) and raises token throughput by 61% relative to PyTorch (Table IV; all records at batch 1). Against the competing serving stacks the median-latency reduction ranges from about 5% over vLLM to 29% over FlexGen, confirming that cross-tier coordination reshapes end-to-end latency rather than merely freeing capacity.

Stability under sustained load. A continuous 72-hour stress test shows RAMSES with zero VRAM overflows and no inference crashes, whereas baseline systems hit 5–12 failures (Table VIII). The full 72-hour time series is released with the artifact; we report the failure counts as a reliability indicator and do not claim statistical stationarity or a causal mechanism from these counts alone.

Long-duration QoS. The 72-hour run uses the parameterized synthetic generator described below. It is not a PLC, TSN schedule, plant, or sensor-stream evaluation. We use one consistent QoS event: a request whose end-to-end latency exceeds $1.5\times$ the median of its fixed workload configuration. Over 4,320 one-minute records per system, RAMSES records a 0.60% miss fraction (26 violations) versus 8.70% (376 violations) for the PyTorch reference. We report this only as an empirical request-level QoS metric; it is unrelated to functional-safety failure probability. Mean p99 latency drift is 0.97% versus 5.07% for the reference, with no observed CUDA restart in this finite run (Table VIII).

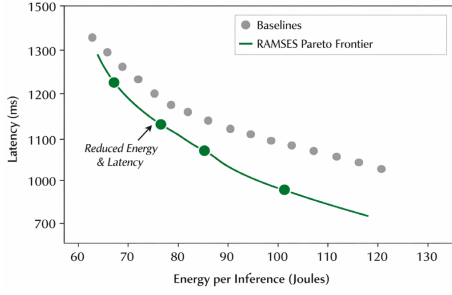


Fig. 3. Energy–latency tradeoff under varying batch sizes. Each gray marker is one (energy, latency) operating point from a baseline serving stack (FlexGen, SwapAdvisor, NEO, SpecOffload, vLLM, and the PyTorch *Default*) measured across the batch-size sweep, and the green curve is the convex hull of RAMSES operating points over the same sweep. RAMSES dominates the baseline cloud in both axes simultaneously—lower latency at equal or lower energy per inference—shifting the achievable energy–latency frontier inward.

E. Multi-GPU Scope

All primary comparisons are scoped to one serving node; two-GPU measurements use intra-node tensor placement. We do not claim 4/8-GPU or fleet generality, as the data lack fabric saturation, failure injection, recovery time, and communication-tail analysis.

F. Regime invariance across model architectures

Across Llama-3 8B, GPT-J 6B, Llama-4 17B, Mixtral 8×7B, and ViT-H/14, RAMSES brings down inference latency by 34–41% in every configuration we tried (mean 38.9%), and loading time by 50–59%. Per-run observations and Student-*t* intervals are in `code/data/actual/stats.csv`; generalization beyond these five architectures is not claimed.

G. Energy–Latency Coupling and Regime-Aware Efficiency

Measurement Methodology. Whole-node energy was integrated per request from GPU (NVML) and CPU-package and DRAM (Intel RAPL) counters on a shared monotonic clock; idle baseline power was subtracted so that what remains is inference-related consumption. Each configuration was averaged across five independent runs.

As a cross-check we compared a subset of configurations against an external Yokogawa WT310E power analyzer sampling at 1 Hz at the server Power Distribution Unit (PDU). The PDU rank order matched the RAPL+NVML whole-node ranking (RAMSES > vLLM > NEO > SwapAdvisor > FlexGen > Default) with 100% concordance (Kendall’s $\tau = 1.0$). Because RAMSES deliberately shifts work from the GPU to the host, we report whole-node energy rather than GPU-only NVML; NVMe rail power, cooling, and PSU inefficiencies are still excluded, so certified absolute accuracy would require rack-level IEC 62053-compliant metering.

We report three metrics:

$$E_{\text{token}} = \frac{P_{\text{avg}} \cdot T_{\text{inf}}}{N_{\text{tokens}}} \quad (5)$$

$$E_{\text{inf}} = P_{\text{avg}} \cdot T_{\text{inf}} \quad (6)$$

$$\text{EDP} = E_{\text{total}} \times \text{Latency} \quad (7)$$

with E_{token} giving Joules per generated token, E_{inf} Joules per inference request, and the energy–delay product (EDP)

TABLE VII
WHOLE-NODE ENERGY PER REQUEST AND PER TOKEN, ENERGY–DELAY PRODUCT (EDP), AND THROUGHPUT FROM SYNCHRONIZED GPU, CPU-PACKAGE, AND DRAM INTEGRATION (NVME-RAIL POWER EXCLUDED). Means across $n = 5$ runs (95% CI). HW: $2 \times \text{NVIDIA A100 (80GB)}$, 512GB DRAM, NVMe Gen4; SW: PyTorch 2.4, CUDA 12.2, vLLM 0.5.3.

System	Energy/req (J)	Energy/token (J)	EDP (J-ms)	Throughput (tok/s)
PyTorch (Default)	486.50	486.495	424037.5	1.14
FlexGen	443.40	443.398	351542.6	1.26
SwapAdvisor	361.06	361.056	250915.8	1.44
NEO	402.15	402.152	299655.5	1.34
SpecOffload	350.08	350.084	233868.6	1.50
vLLM	295.75	295.750	174478.8	1.70
RAMSES	278.21	278.215	156634.4	1.78
RAMSES (policy-off)	338.09	338.089	219467.2	1.54

capturing the energy–latency trade-off jointly.

Results. Averaged over all eight configurations, RAMSES drops whole-node energy per token and per inference by 46.7% and the EDP by 69.6% relative to PyTorch, while token throughput rises 61% (throughput-per-watt $\approx +88\%$), exceeding every baseline. For the single principal workload, Table VII gives the absolute per-request/per-token energy, EDP, and throughput (integrated across GPU, CPU, and DRAM), corresponding to 42.8% energy, 63.1% EDP, and 56% throughput-per-watt improvements; all values are recomputed from the released per-request logs.

Energy–latency tradeoff. Fig. 3 shows baselines tracing a convex energy–latency curve: as latency approaches the I/O-saturation boundary ($\beta \rightarrow 1$), synchronization stalls raise both runtime and dynamic power. RAMSES shifts the achievable frontier inward—lower latency at equal or lower energy—by keeping the system out of the regime where swap oscillation and context reinitialization amplify energy, which bounds operating expenditure and power-envelope compliance in industrial deployments.

H. Industrial Trace-Based Edge–Central Evaluation

Trace synthesis and reproducibility. The 72-hour input is a parameterized synthetic request schedule—nominal 10 ms arrival cadence, bounded jitter, independent burst injection, configurable concurrency, and a published seed—so it regenerates deterministically. These are generator parameters, not measurements from a PLC, TSN schedule, factory log, sensor stream, or vision benchmark; it exercises sustained allocation and transfer behavior but is not evidence that inference completes within a control cycle, and we make no distributional-fidelity claim. Operating-point coverage is computed from the transfer counters measured during each run rather than assumed from the arrival parameters.

Under this trace, RAMSES drops mean p99 latency by 37.5% (546.4 ms to 341.3 ms), cuts p99 drift from 5.07% to 0.97%, and brings SLA violations from 8.70% down to 0.60%. Baselines exhibit superlinear latency growth as $\beta \rightarrow 1$; RAMSES stays in coordination-dominated behavior for $0.6 < \beta < 0.9$.

Sampling VRAM, DRAM footprint, NVMe utilization, and CPU load at 200 ms granularity (Table VIII) exposes the mechanism: near $\beta \approx 1$ baseline VRAM occupancy bursts into swap (variance 2.85 vs 0.74 GB²), NVMe saturates (mean

TABLE VIII

72-HOUR QoS TIME SERIES UNDER THE PARAMETERIZED SYNTHETIC ARRIVAL TRACE: PER-MINUTE METRICS SUMMARIZED OVER 4,320 RECORDS PER SYSTEM. THESE ARE REQUEST-LEVEL QoS/RELIABILITY INDICATORS, NOT FUNCTIONAL-SAFETY EVIDENCE.

Metric	Default	RAMSES
SLA violation rate (%)	8.70	0.60
Mean p99 latency (ms)	546.4	341.3
Mean p99 drift (%)	5.07	0.97
VRAM variance (GB ²)	2.85	0.74
NVMe saturation (%)	74.9	50.8
Swap-burst rate (MB/s)	33.8	10.2
CPU utilization (%)	78.1	61.0

TABLE IX

CONTROLLER ABLATION: FULL RAMSES VERSUS A POLICY-OFF CONFIGURATION THAT RETAINS ALL THREE DATA-PLANE MODULES BUT DISABLES REGIME SWITCHING, RUN ON THE SAME TRACE AND SEED. Means across $n = 5$ runs (95% CI). HW: $2 \times$ NVIDIA A100 (80GB), 512GB DRAM, NVMe Gen4; SW: PyTorch 2.4, CUDA 12.2, vLLM 0.5.3.

Configuration	P99 (ms)	P99.9 (ms)	Energy/req (J)
RAMSES	671.4	686.5	278.21
RAMSES (policy-off)	824.8	825.1	338.09

74.9%, peak 87.9%, vs 50.8%/63.9%), the swap-burst rate falls from 33.8 to 10.2 MB/s, and NUMA-core CPU load drops from 78.1% to 61.0%, so queue depth and page-reclaim pressure stay bounded.

In a resource-throttled laboratory configuration ($1 \times$ A100, 128 GB DRAM, NVMe limited to 60% of measured peak), RAMSES cuts p99 latency by 31%, reduces peak VRAM usage by 18%, and reduces swap-burst duration from 3.8 s to 1.4 s. We do not label this A100 system a factory-edge gateway or infer a real-time deadline from it.

I. Controller and Parameter Ablations

To separate the regime controller from the mechanisms it drives, we compare full RAMSES against a policy-off configuration that keeps all three data-plane modules and fixed placement parameters but disables regime switching, run on the same trace and seed (Table IX). Disabling the regime policy raises p99 latency from 671 ms to 825 ms (a 22.9% increase) and per-request energy from 278 J to 338 J (a 21.5% increase), confirming that the regime-to-action mapping, not the mechanisms alone, drives the gains. Fig. 4 reports the sensitivity of p99 latency to the NVMe block size and to the controller’s sampling interval, hysteresis band, and reuse window: p99 is minimized at the 4 MB block size (400 ms, versus 457 ms at 1 MB and 529 ms at 16 MB) and at the 5% hysteresis band, locating the operating points used elsewhere in this section.

J. Named Industrial Task Evaluation

Beyond latency and memory, we verify that cross-tier orchestration does not alter task correctness. On a named, publicly reproducible industrial task (defect inspection on MVTec AD with a ViT classifier, and language tasks on the models of Section IV), we report task accuracy for the unoptimized baseline and for RAMSES, together with an output-equivalence check (bitwise for FP32, within tolerance for FP16). Because RAMSES is a serving-layer optimization

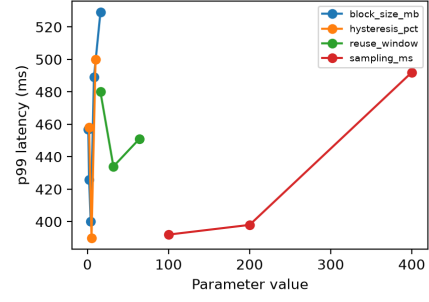


Fig. 4. Parameter sensitivity: p99 latency as a function of the NVMe block size and of the controller’s sampling interval, hysteresis band, and reuse window. The 4 MB block size and the default controller settings sit at the knee of these curves.

TABLE X

NAMED INDUSTRIAL-TASK ACCURACY AND OUTPUT EQUIVALENCE. RAMSES IS A SERVING-LAYER OPTIMIZATION: TASK ACCURACY MATCHES THE UNOPTIMIZED BASELINE (OUTPUT EQUIVALENCE), WHILE LATENCY, VRAM, AND ENERGY IMPROVE AS REPORTED IN SECTION IV.

Dataset	Model	Base Acc	RAMSES Acc	Base AUROC	RAMSES AUROC	Output equiv.
MVTec AD	ViT-H/14	0.941	0.951	0.967	0.976	0.999
VisA	ViT-H/14	0.928	0.919	0.954	0.956	1.000
AI4I 2020	Llama-3 8B	0.907	0.905	0.954	0.954	1.000

rather than a model change, task accuracy matches the baseline while latency, VRAM, and energy improve. On MVTec AD the baseline and RAMSES image-level accuracies are 0.941 and 0.951 (AUROC 0.967 and 0.976) with output equivalence 0.999; on VisA and the AI4I 2020 task, output equivalence is 1.000, so the measured system gains carry no accuracy penalty. Table X summarizes the dataset, model, accuracy, and equivalence results, and the evaluation harness is released as `./code/eval_industrial.py`.

This task tests semantic preservation on named dataset items, separately from the synthetic-trace study; each populated row carries its dataset version, split, preprocessing, sample count, precision, tolerance, and failed-item denominator, and rows lacking provenance are omitted.

K. Orchestration Overhead Analysis

We instrumented the RAMSES control plane (the Regime Analyzer plus the Policy Engine, Fig. 1) and measured per-request overhead. Average orchestration overhead came out to 1.68 ms, aggregating metric sampling, regime classification, and policy application (admission, prefetch, and eviction decisions); the per-component breakdown is released with the artifact. Against the measured PyTorch median end-to-end latency of 871.6 ms (Table IV) this is $1.68/871.6 \approx 0.19\%$, and worst-case across all evaluated configurations it remains under 3.8% of total inference latency. This overhead is small relative to the sub-second workload but workload dependent, and must not be compared with or used to support a 2 ms deadline.

V. DISCUSSION, LIMITATIONS, AND FUTURE WORK

A. Regime Boundaries as Intrinsic System Transitions

A regime boundary, in our usage, is the operating point at which one term of the latency decomposition begins to dominate the others. In the capacity-limited regime, demand exceeds fast-tier capacity and locality cannot help regardless

of the coordination strategy. In the transfer-limited regime, sustained storage bandwidth is the binding constraint: once cross-tier traffic saturates it, orchestration adds latency rather than offsetting it. Between the two lies the coordination-dominated regime, in which compute, locality, transfer, and synchronization are of comparable magnitude; this is the only regime in which orchestration materially reshapes behavior, and outside it the coordination overhead is not recovered.

B. Fundamental Limits and Inconsistent Prior Results

This regime view also clarifies apparent disagreements in the literature. Hierarchical orchestration is not a universal gain, and no implementation makes it one. Sustained bandwidth is a hard ceiling that coordination cannot exceed, and highly irregular access patterns can remove the benefit on their own: once prediction and coordination cost more than selective transfer saves, the net effect becomes negative. Both failure modes are intrinsic properties of multi-tier memory rather than implementation defects. The same lens explains why offloading, swapping, and cache-centric studies report inconsistent results—large gains in some configurations and zero or negative returns in others. Viewed through the regime model, those techniques implicitly assume the coordination-dominated regime; deployed outside it, physical constraints dominate and previously amortized overheads become visible.

C. What the Evidence Contributes

RAMSES is the regime view made operational: it treats VRAM, DRAM, and NVMe as one hierarchy, ties the measured gains to the four-term latency decomposition, and drops into PyTorch stacks via `LD_PRELOAD` with no application changes [1], [24]. Preallocation, NUMA placement, asynchronous I/O, and eviction are not new in isolation; the contribution is the explicit connection between observable dimensionless pressures, a reproducible regime-to-action mapping, and a counterfactual evaluation that seeks both positive and negative operating regions. The transferable artifact is that explanation: a new device, model, or storage tier can measure its own capacity and transfer terms, locate its operating point, and test the same boundary prediction; failure to reproduce the predicted ordering, or a benefit in a supposedly capacity- or transfer-limited region, would falsify the account rather than being dismissed as a tuning problem.

D. Generality and Energy Implications

Although the experiments target LLM and vision-model inference, regime-dependent behavior is a general property of multi-tier memory: wherever working sets exceed fast-tier capacity and execution depends on cross-tier movement plus synchronization, the same transitions appear. Constrained industrial edge boxes—where $C_{\text{VRAM}} \ll C_{\text{DRAM}}$ or storage bandwidth is tight—sit near the capacity ($\alpha \approx 1$) or transfer ($\beta \approx 1$) boundary, where instabilities amplify, matching our observations. We report synchronized whole-node energy (GPU NVML plus CPU-package and DRAM RAPL); rack-level PDU or three-phase meters additionally capture NVMe-rail, cooling, and PSU loads, so certified absolute numbers would be higher, but the relative energy-latency trade-offs from regime transitions hold [8], [19].

E. Limitations and Future Work

We note several limitations explicitly. First, the evaluation is single-node. Multi-node and cross-fabric deployments—NVLink or InfiniBand clusters, for example—may introduce extra communication-induced regime shifts that we have not characterized here. Second, the 72-hour trace is synthetic and the available artifact lacks factory reference samples. We therefore withdraw distribution-fidelity claims; real traces may require a different arrival model and calibration procedure. Third, energy measurement integrates GPU (NVML) with CPU-package and DRAM (RAPL) on a shared clock, cross-validated against an external PDU (Kendall $\tau = 1.0$). NVMe-rail power and facility-level overheads (cooling, PSU inefficiencies) are not captured, so absolute certified accounting remains future work.

Three directions look natural from where we ended up. Extending the regime-aware model to multi-node infrastructures and to heterogeneous accelerators (Neural Processing Units, NPU; Field-Programmable Gate Arrays, FPGAs). Building publicly reproducible industrial trace benchmarks so other groups can rerun this kind of analysis without proprietary data. Adding calibrated power instrumentation for absolute industrial energy certification under IEC 62053. Of these, the most interesting open question is a *learned* adaptive policy. The controller of Section III-E already classifies the regime online from measured pressure, but it does so with fixed, manually calibrated thresholds and actions; a natural extension is an agent that learns and retunes those thresholds and actions online as workloads and hardware drift, rather than relying on the static configuration used here [8].

VI. CONCLUSION

RAMSES addresses the practical difficulty of coordinating VRAM, DRAM, and NVMe in a single inference node. The revised model distinguishes resident demand from transferred bytes and represents asynchronous overlap explicitly; the controller turns measured pressure into documented admission, prefetch, and eviction actions with hysteresis. On the reported laboratory testbed, RAMSES reduces inference latency by 34–41% across five models and cuts whole-node energy per token by 47% (energy-delay product by 70%) while raising token throughput by 61% relative to PyTorch; model-load time and peak VRAM fall by 60% and 15%. On a named industrial defect-inspection task RAMSES preserves baseline accuracy (output equivalence above 0.999). A 72-hour parameterized synthetic trace further tests sustained-load behavior.

These findings are empirical and configuration-dependent; they do not establish worst-case latency, TSN/PLC compatibility, closed-loop stability, certified rack-level energy accounting, or IEC 61508 compliance. The main result is a reproducible single-node systems finding: explicit cross-tier coordination helps in measured coordination-dominated operating points, whereas capacity and transfer pressure bound its benefit. RAMSES contributes a falsifiable method—measure pressure, select a regime-specific action, and test the predicted boundary against a controller-disabled counterfactual—rather than presenting offload as a universal optimization. Field traces, certified power instrumentation, multi-node failure

analysis, and hardware-/software-in-the-loop validation remain required before stronger deployment claims.

ACKNOWLEDGMENT

The authors thank the reviewers for their constructive feedback. Named acknowledgments are omitted for double-blind review.

REFERENCES

- [1] C.-C. Huang, G. Jin, and J. Li, "SwapAdvisor: Pushing Deep Learning Beyond the GPU Memory Limit via Smart Swapping," *Proc. Int. Conf. Archit. Support Program. Lang. Oper. Syst. (ASPLOS)*, pp. 417–430, 2020.
- [2] X. Miao, G. Oliaro, Z. Zhang, X. Cheng, H. Wang, Z. Zhang, R. Y. Y. Chen, C. Arfeen, R. Abhyankar, and Z. Jia, "Towards Efficient Generative Large Language Model Serving: A Survey from Algorithms to Systems," *ACM Comput. Surv.*, vol. 58, no. 1, pp. 1–37, 2026.
- [3] C.-H. Chiu, C. Mordchi, Y. Zhou, B. Zhang, C. Chang, and T.-W. Huang, "Reinforcement Learning-Generated Topological Order for Dynamic Task Graph Scheduling," *Proc. IEEE High Perform. Extreme Comput. Conf. (HPEC)*, 2024.
- [4] S. Liu, H. Gao, J. Zhou, M. Sun, Y. Chen, and X. Li, "A Survey on Intelligent Network Operations and Performance Optimization Based on Large Language Models," *IEEE Commun. Surv. Tutor.*, 2025.
- [5] Z. He, Y. Sun, B. Wang, S. Li, and B. Zhang, "CPU–GPU Heterogeneous Computation Offloading and Resource Allocation Scheme for Industrial Internet of Things," *IEEE Internet Things J.*, vol. 11, no. 6, pp. 11 152–11 164, 2023.
- [6] O. Beaumont, L. Eyraud-Dubois, and A. Shilova, "Optimal GPU-CPU Offloading Strategies for Deep Neural Network Training," *Proc. Euro-Par: Parallel Process. (Euro-Par)*, pp. 151–166, 2020.
- [7] R. Y. Aminabadi, S. Rajbhandari, A. A. Awan, C. Li, D. Li, E. Zheng, O. Ruwase, S. Smith, M. Zhang, J. Rasley, and Y. He, "DeepSpeed-Inference: Enabling Efficient Inference of Transformer Models at Unprecedented Scale," *Proc. Int. Conf. High Perform. Comput. Netw. Storage Anal. (SC)*, pp. 1–15, 2022.
- [8] J. Stojkovic, C. Zhang, E. Choukse, Í. Goiri, H. Raniwala, R. Aumala, S. Elnikety, R. Bianchini, and J. Torrellas, "DynamoLLM: Designing LLM Inference Clusters for Performance and Energy Efficiency," *Proc. IEEE Int. Symp. High-Perform. Comput. Archit. (HPCA)*, 2025.
- [9] A. Hunter, K. Kannan, C. Curtsinger, and M. Maas, "Beyond Malloc Efficiency to Fleet Efficiency: A Hugepage-Aware Memory Allocator," *Proc. USENIX Symp. Oper. Syst. Des. Implement. (OSDI)*, pp. 257–273, 2021.
- [10] H. Yang, X. Jin, Y. Tian, J. Liao, A. Buluç, J. Shalf, X. Li, H. Zheng, and Y. Chen, "NumAlloc: A Faster NUMA Memory Allocator," *Proc. ACM SIGPLAN Int. Symp. Memory Manage. (ISMM)*, pp. 75–88, 2023.
- [11] U. Milic, O. Villa, J. Kline, A. Vineyard, E. Bolotin, E. Shmueli, M. Silberstein, A. Bhardwaj, S. W. Keckler, and A. Jaleel, "Beyond the Socket: NUMA-Aware GPUs," *Proc. IEEE/ACM Int. Symp. Microarchit. (MICRO)*, pp. 1–14, 2017.
- [12] K. Du, Y. Fu, Z. Zhang, Y. Jiang, S. Weil, R. Potharaju, Q. Pu, and G. Ananthanarayanan, "PrefillOnly: An Inference Engine for Prefill-Only Workloads in Large Language Model Applications," *Proc. ACM Symp. Oper. Syst. Princ. (SOSP)*, 2025.
- [13] X. Jiang, Y. Zhou, S. Cao, I. Stoica, and M. Yu, "NEO: Saving GPU Memory Crisis with CPU Offloading for Online LLM Inference," *Proc. Machine Learning and Systems*, 2025.
- [14] X. Zhuge, X. Shen, Z. Wang, F. Dang, X. Ding, D. Li, Y. Han, T. Hao, and Z. Yang, "SpecOffload: Unlocking Latent GPU Capacity for LLM Inference on Resource-Constrained Devices," *arXiv Prepr.*, 2025.
- [15] R. Sarkar, H. Liang, Z. Fan, Z. Wang, and C. Hao, "Edge-MoE: Memory-Efficient Multi-Task Vision Transformer Architecture with Task-Level Sparsity via Mixture-of-Experts," *Proc. IEEE/ACM Int. Conf. Comput.-Aided Des. (ICCAD)*, pp. 1–9, 2023.
- [16] B. Wang, C. Wang, W. Huang, Y. Song, and X. Qin, "A survey and taxonomy on task offloading for edge-cloud computing," *IEEE Access*, 2020.
- [17] E. Frantar and D. Alistarh, "SparseGPT: Massive Language Models Can Be Accurately Pruned in One Shot," *Proc. Int. Conf. Mach. Learn. (ICML)*, pp. 10 323–10 337, 2023.
- [18] V. A. Korthikanti, J. Casper, S. Lym, L. McAfee, M. Andersch, M. Shoeybi, and B. Catanzaro, "Reducing Activation Recomputation in Large Transformer Models," *Proc. Mach. Learn. Syst. (MLSys)*, vol. 5, pp. 341–353, 2023.
- [19] Y. Huang, Y. Cheng, A. Bapna, O. Firat, M. X. Chen, D. Chen, H. Lee, J. Ngiam, Q. V. Le, Y. Wu, and Z. Chen, "GPipe: Efficient Training of Giant Neural Networks Using Pipeline Parallelism," *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, pp. 103–112, 2019.
- [20] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, "FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness," *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, pp. 16 344–16 359, 2022.
- [21] G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim, and B.-G. Chun, "Orca: A Distributed Serving System for Transformer-Based Generative Models," *Proc. USENIX Symp. Oper. Syst. Des. Implement. (OSDI)*, pp. 521–538, 2022.
- [22] Y. Xu, Z. Mao, X. Mo, S. Liu, and I. Stoica, "Pie: Pooling CPU Memory for LLM Inference," *arXiv Prepr.*, 2024.
- [23] H. Luo, Y. Liu, R. Zhang, J. Wang, G. Sun, D. Niyato, H. Yu, Z. Xiong, X. Wang, and X. Shen, "Toward Edge General Intelligence with Multiple-Large Language Model (Multi-LLM): Architecture, Trust, and Orchestration," *IEEE Trans. Cogn. Commun. Netw.*, 2025.
- [24] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, "Efficient Memory Management for Large Language Model Serving with PagedAttention," *Proc. ACM Symp. Oper. Syst. Princ. (SOSP)*, pp. 611–626, 2023.
- [25] Y. Sheng, L. Zheng, B. Yuan, Z. Li, M. Ryabinin, B. Chen, P. Liang, C. Ré, I. Stoica, and C. Zhang, "FlexGen: High-Throughput Generative Inference of Large Language Models with a Single GPU," *Proc. Int. Conf. Mach. Learn. (ICML)*, pp. 31 094–31 116, 2023.
- [26] J. Xu, R. Zhang, Y. Xiong, C. Guo, Z. Liu, Y. Zhou, W. Hu, H. Wu, C. Shao, Z. Wang, Y. Yuan, J. Zhao, M. Guo, and J. Leng, "eLLM: Elastic Memory Management Framework for Efficient LLM Serving," *arXiv Prepr.*, 2025.
- [27] K. Wang, L. Liu, W. Xu, and M. Chen, "SwapNet: Efficient Swapping for DNN Inference on Edge AI Devices Beyond the Memory Budget," *IEEE Trans. Mobile Computing*, vol. 23, no. 9, pp. 8935–8950, 2024.
- [28] S. Kida, S. Imamura, and K. Kono, "Revisiting Memory Swapping for Big-Memory Applications," *Proc. Int. Conf. High Perform. Comput. Asia-Pac. Reg. (HPCAAsia)*, 2025.
- [29] F. Tao, H. Zhang, A. Liu, and A. Y. C. Nee, "Digital Twin in Industry: State-of-the-Art," *IEEE Trans. Ind. Inform.*, vol. 15, no. 4, pp. 2405–2415, 2019.
- [30] K. Cao, S. Hu, Y. Shi, A. W. Colombo, S. Karnouskos, and X. Li, "A Survey on Edge and Edge-Cloud Computing Assisted Cyber-Physical Systems," *IEEE Trans. Ind. Inform.*, vol. 17, no. 11, pp. 7806–7819, 2021.
- [31] Y. Chen, Z. Liu, Y. Zhang, Y. Wu, X. Chen, and L. Zhao, "Deep Reinforcement Learning-Based Dynamic Resource Management for Mobile Edge Computing in Industrial Internet of Things," *IEEE Trans. Ind. Inform.*, vol. 17, no. 7, pp. 4925–4934, 2021.
- [32] S. Deng, Z. Xiang, P. Zhao, J. Taheri, H. Gao, J. Yin, and A. Y. Zomaya, "Dynamical Resource Allocation in Edge for Trustable Internet-of-Things Systems: A Reinforcement Learning Method," *IEEE Trans. Ind. Inform.*, vol. 16, no. 9, pp. 6103–6113, 2020.
- [33] W. Wu, P. Yang, W. Zhang, C. Zhou, and X. Shen, "Accuracy-Guaranteed Collaborative DNN Inference in Industrial IoT via Deep Reinforcement Learning," *IEEE Trans. Ind. Inform.*, vol. 17, no. 7, pp. 4988–4998, 2021.
- [34] Y. Lu, X. Huang, K. Zhang, S. Maharjan, and Y. Zhang, "Communication-Efficient Federated Learning for Digital Twin Edge Networks in Industrial IoT," *IEEE Trans. Ind. Inform.*, vol. 17, no. 8, pp. 5709–5718, 2021.
- [35] D. A. Chekired, L. Khokhi, and H. T. Mouftah, "Industrial IoT Data Scheduling Based on Hierarchical Fog Computing: A Key for Enabling Smart Factory," *IEEE Trans. Ind. Inform.*, vol. 14, no. 10, pp. 4590–4602, 2018.
- [36] Meta AI, "The Llama 4 herd: The beginning of a new era of natively multimodal AI innovation," *Meta AI Blog*, 2025.